COS30017. SOFTWARE DEVELOPMENT FOR MOBILE DEVICES

Dr. Nguyen Dang Khoa
Email: kdnguyen@swin.edu.au
Phone: 0974.844.618

class LoginActivity

```
lateinit var btnLogin: Button
lateinit var txtUser: EditText
lateinit var txtPass: EditText

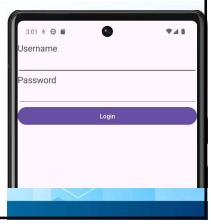

btnLogin= findViewById(R.id.btnLogin)
txtUser = findViewById(R.id.txtUser)
txtPass = findViewById(R.id.txtPass)
btnLogin.setOnClickListener(this)


override fun onClick(v: View?) {
   when (v?.id) {
      R.id.btnLogin -> {
         if ((txtUser.text.toString()=="KhoaND") && (txtPass.text.toString()=="123"))
         {
            //Toast.makeText(this, "Correct", Toast.LENGTH_LONG).show()
            val intent = Intent(this, AdminActivity::class.java)
            startActivity(intent)
         }
         else
            Toast.makeText(this, "Not correct", Toast.LENGTH_LONG).show()
      }
   }
}
```

AndroidManifest.xml

```
<activity
   android:name=".LoginActivity"
   android:exported="true">
   <intent-filter>
      <action android:name="android.intent.action.MAIN" />
      <category android:name="android.intent.category.LAUNCHER" />
   </intent-filter>
</activity>

<activity android:name=".AdminActivity"></activity>
```



Week 4 – Call activity (Intent)

LoginActivity.kt

```
val intent = Intent(this, AdminActivity::class.java)
intent.putExtra("Username", txtUser.text.toString())

startActivity(intent)
```

AdminActivity.kt

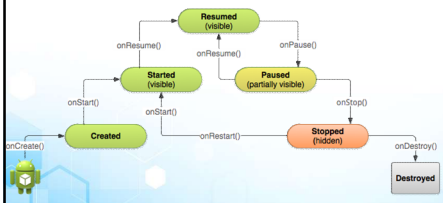
```
override fun onCreate(savedInstanceState: Bundle?) {
   super.onCreate(savedInstanceState)
   enableEdgeToEdge()
   setContentView(R.layout.adminlayout)

   //.....
   val intent = getIntent()
   val value = intent.getStringExtra("Username")

Toast.makeText(this, value, Toast.LENGTH_LONG).show()

}
```

Week 4 –Activity Lifecycles and State



- onCreate(), called when the system creates the activity. This is where the layout for the UI should be defined.
- onStart(), called when the activity becomes visible to the user.
- onResume(), called when the activity is at the top of the activity stack.
- onPause(), called when the activity loses focus.
- onStop(), called when the activity is no longer visible to the user.
- onRestart(), called when a stopped activity will restart.
- onDestroy(), called just before the activity is destroyed.

Week 4 –Activity Lifecycles and State

- Write a log and test all

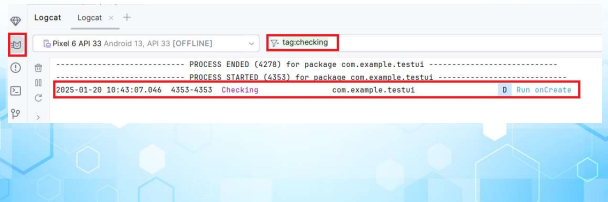
```
class LoginActivity: AppCompatActivity() , View.OnClickListener {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.loginlayout)
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
            insets
        }
        Log.d( tag: "Checking",  msg: "Run onCreate");
    }
```

Log.d("Checking", "Run onCreate");

Week 4 –Activity Lifecycles and State

- Write a log and test all

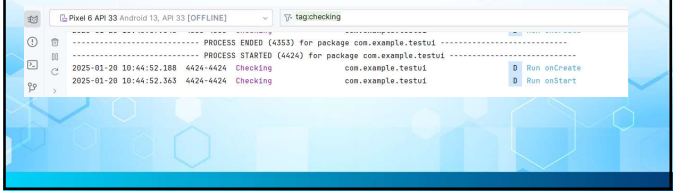
Log.d("Checking", "Run onCreate");



Week 4 –Activity Lifecycles and State

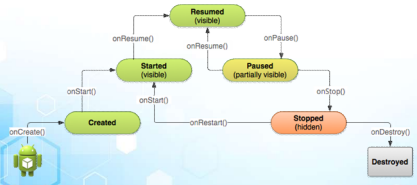
- Write a log and test all

```
public override fun onStart() {
    super.onStart()
    Log.d("Checking", "Run onStart")
}
```



Week 4 –Activity Lifecycles

- Write a report and upload to github about process



onSaveInstanceState



LoginActivity.kt

```
public override fun onStart() {
    super.onStart()
    Log.d("Checking", "OnStart running")
}
override fun onPause() {
    super.onPause()
    Log.d("Checking", "onPause")
}
override fun onRestart() {
    super.onRestart()
    Log.d("Checking", "onRestart")
}
override fun onResume() {
    super.onResume()
    Log.d("Checking", "onResume")
}
override fun onStop() {
    super.onStop()
    Log.d("Checking", "onStop")
}
override fun onDestroy() {
    super.onDestroy()
    Log.d("Checking", "onDestroy")
}
```

Run program

```
D OnCreate running
D OnStart running
D onResume
```

Login click

```
D onPause
D onStop
```

Exit click

```
D End Admin
D onRestart
D OnStart running
D onResume
```

AdminActivity.kt

```
lateinit var txtAdmin: TextView
lateinit var btnExit: Button

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    //....
    val intent = getIntent()
    val value = intent.getStringExtra("Username")
    Toast.makeText(this, value, Toast.LENGTH_LONG).show()

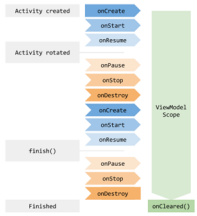
    txtAdmin = findViewById(R.id.txtAdmin)
    txtAdmin.setText("Hello " + value)

    btnExit= findViewById(R.id.btnExit)
    btnExit.setOnClickListener(this)

}

override fun onClick(v: View?) {
    when (v?.id) {
        R.id.btnExit -> {
            Log.d("Checking", "End Admin");
            finish();
        }
    }
}
```

Week 4 – ViewModel to save state



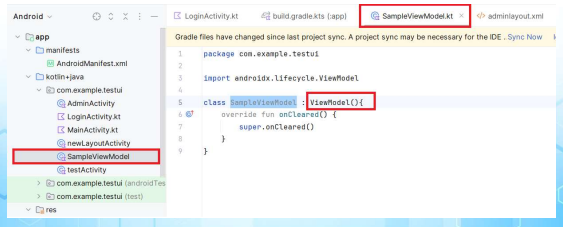
- Add dependencies to app/build.gradle

implementation ("androidx.lifecycle:lifecycle-viewmodel-ktx:2.6.1")
implementation ("androidx.activity:activity-ktx:1.7.2")



Week 4 – ViewModel to save state

- Create the SampleViewModel

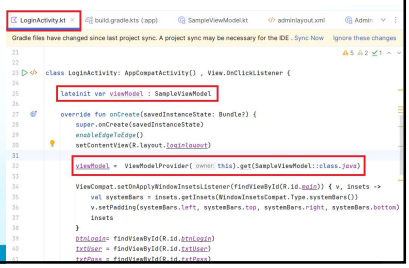


Week 4 – ViewModel to save state

- Add SampleViewModel to MainActivity (this case is LoginActivity)

lateinit var viewModel : SampleViewModel

viewModel = ViewModelProvider(this).get (SampleViewModel::class.*java*)



Week 4 – ViewModel to save state

- Add SampleViewModel to MainActivity (this case is LoginActivity)

SampleViewModel.kt

```
import androidx.lifecycle.LiveData
import androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModel

class SampleViewModel : ViewModel(){

    private val _count = MutableLiveData<Int>()
    var number = 0

    val badgeCount: LiveData<Int>
        get() = _count

    fun incrementBadgeCount() {
        _count.postValue(++number)
    }

    override fun onCleared() {
        super.onCleared()
    }
}
```

LoginActivity.kt

```
lateinit var viewModel: SampleViewModel
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()
    setContentView(R.layout.loginlayout)

    viewModel = ViewModelProvider(this).get(SampleViewModel::class.java)
    observeViewModel()

private fun observeViewModel() {
    viewModel.badgeCount.observe(this, Observer {
        showToast(it)
    })
}
private fun showToast(value: Int) {
    Toast.makeText(this, value.toString(), Toast.LENGTH_LONG).show()
}

when (v?.id) {
        R.id.btnLogin -> {
            if ((txtUser.text.toString()=="KhoaND") && (txtPass.text.toString()=="123"))
                viewModel.incrementBadgeCount()
        }
```

Week 4 – Architecture (MVC)

- M: the model is a data structure that takes input from the controller and updates the view
- V: the view is a representation of the data (e.g., a UI) which is seen by the user
- C: the controller handles input/actions from the user and updates the model as needed.